LRN · LAB REVENUE NAVIGATOR

BeechTree Executive Summary Three-Pillar Diagnostics Two-Agent Pipeline

Microsoft Foundry design — JSON generation and model insights

PLANNING DOCUMENT

Agent 1 · Generate Three-Pillar JSON
Agent 2 · Generate insights using Foundry models
Trigger · New Executive Summary refresh + weekly schedule
16 August 2026

Internal use · Includes table of contents and term index

Contents

This document is the implementation plan for a scheduled two-agent pipeline: Agent 1 generates BeechTree Three-Pillar diagnostic values as JSON; Agent 2 uses Microsoft Foundry models to produce insights. Use the Contents below to open a chapter. Section 11 is an alphabetical index of tools and terms with context.

1. Purpose and outcome

The goal is a production pipeline that runs when a new BeechTree Executive Summary is generated, exports the Three-Pillar diagnostic values to a JSON file, and immediately hands that file to a second agent that writes insights using a Foundry-hosted model. A weekly schedule is a safety net if the event is missed.

Intended outcome

- Agent 1 produces one JSON file whose LIS, PMS, and Cash numbers match the Three-Pillar Diagnostic page.
- Agent 2 starts only after that JSON exists and validates — the file is the contract between the agents.
- Insights are stored (Blob plus Executive Summary Notes) so operators can review them with the weekly report.
- Microsoft Foundry supplies models for Agent 2; Azure Logic Apps is the conductor (schedule + event + sequence).

> *Do not combine both jobs in one agent, and do not use Foundry's visual workflow designer for this build. Microsoft is retiring that designer (1 December 2026). Use Logic Apps Standard to orchestrate Foundry agents.*

2. Recommended architecture

Treat Agent 1 as a deterministic exporter and Agent 2 as the model. Logic Apps receives the Executive Summary refresh event, calls Agent 1, validates the Blob JSON, then runs Agent 2. This keeps billed dollars and claim counts exact, and spends model tokens only on interpretation.

Handoff rule

Agent 2 must not run on its own timer against stale or missing data. It starts only when Agent 1 has written a valid JSON object containing **lis**, **pms**, and **cash** sections.

Why not scrape the dashboard

The Three-Pillar page already loads from three stored procedures on BeechTree_LRN. Agent 1 should call those procedures (or a thin JSON API that wraps them). HTML scraping would drift from the page and is slower to operate.

What already exists in LRN

Pillar	Stored procedure	Dashboard load
LIS — sample to claim	usp_GetBeechTree_ThreePillarLisDiagnostic	Main Three-Pillar page
PMS — revenue realization	usp_GetBeechTree_ThreePillarPmsDiagnostic	PMS tab (lazy load)
Cash — leakage and risk	usp_GetBeechTree_ThreePillarCashDiagnostic	Cash tab (lazy load)

Page today: /ExecutiveSummary/ThreePillarDiagnostic?lab=Beech_Tree&months=12. Shared window: trailing months (3 / 6 / 9 / 12 / 19) plus comparable day cutoff from WeekRange end. Executive Summary aggregates

already refresh in ClaimLineCSVDataCapture after new lab data.

3. Process flowchart

Read the table top to bottom. Each stage waits for the previous stage. The JSON Blob between stages 3 and 5 is the only payload Agent 2 is allowed to reason over.

#	Stage	What happens
1	**SOURCE EVENT**	ClaimLineCSVDataCapture — BeechTree Executive Summary refresh succeeds (STEP 16 new RunId, or STEP 14 TransactionDetail).
2	**EVENT + SCHEDULE**	HTTP webhook or Service Bus message to Logic App. Optional Monday weekly recurrence as a safety net.
3	**AGENT 1 — JSON**	Azure Function calls LIS, PMS, and Cash stored procedures. Writes one JSON file to Blob Storage.
4	**VALIDATE HANDOFF**	Logic App checks that lis, pms, and cash sections are present. Fail and alert if not.
5	**AGENT 2 — INSIGHTS**	Foundry model reads the JSON and diagnostic playbook. Emits structured findings (risk, evidence, action).
6	**PERSIST + NOTIFY**	Save insights JSON to Blob. Insert rows via usp_NotesInsight_Insert. Optional email / Teams.

Figure 1. End-to-end BeechTree Three-Pillar agent pipeline (event-driven, with weekly backup).

Trigger sources on the Logic App

Trigger	When it fires	Why it exists
ES refresh event (primary)	ClaimLineCSVDataCapture STEP 16 (new RunId) or STEP 14 (new TransactionDetail file), after BeechTree ES SPs succeed	Insights follow the real weekly Executive Summary
Weekly recurrence (backup)	For example Monday 07:00	Catches a missed webhook; Agent 2 still waits for fresh JSON

4. Agent responsibilities

	Agent 1 — JSON Generator	Agent 2 — Insights
Job	Pull Three-Pillar numbers	Interpret those numbers
Brain	No LLM. Deterministic code (Azure Function)	Foundry-deployed chat model
Input	Lab Beech_Tree, months, as-of / day window	The JSON file from Agent 1 plus a diagnostic playbook
Output	One JSON file in Blob Storage	Insights JSON and Notes rows
Foundry role	Optional OpenAPI wrapper around the Function	Primary agent: instructions, model, structured output

	Agent 1 — JSON Generator	Agent 2 — Insights
Why split	Numbers must match the dashboard	Wording and diagnosis are the model's job

> *Agent 1 must not use a language model to invent metrics. Registering the Function as a Foundry OpenAPI tool is optional and only for catalog/visibility. The Function remains the source of truth.*

Agent 1 JSON contract (sketch)

```
{
  "meta": { "lab": "Beech_Tree", "trailingMonths": 12,
          "asOfDate": "...", "dayWindow": 23, "weekFolder": "..." },
  "lis":  { "monthly": [], "funnelPeriod": {}, "backlogSummary": {}, "backlogBuckets": [] },
  "pms":  { "reconciliation": [], "fullyAdjusted": [], "fullyPaid": [], "denialByCarrier": [] },
  "cash": { "headline": [], "writeOffReasons": [] }
}
```

Shape should follow ExecSummaryThreePillarViewModel so a reviewer can compare JSON to the page field-for-field. Blob path example: beech-tree/three-pillar/{weekFolder}/three-pillar.json.

Agent 2 insights contract (sketch)

```
{
  "lab": "Beech_Tree", "weekFolder": "...",
  "findings": [{
    "pillar": "LIS", "risk": "Red", "title": "...",
    "insight": "...", "evidence": "cite JSON fields",
    "action": "...", "responsibleParty": "Billing"
  }]
}
```

Use Foundry structured outputs so every run returns this schema. Agent 2 instructions: do not invent metrics; cite evidence from the JSON; classify risk as Red / Yellow / Green; propose a concrete action and owner.

5. Tools required

5.1 Azure platform

Tool	Role in this pipeline
Azure subscription	Host all resources
Microsoft Foundry (project + model deployment)	Agent 2 models and agent definition
Azure Logic Apps (Standard)	Event + schedule + Agent 1 then Agent 2 sequence
Azure Functions	Agent 1: call three SPs, write JSON
Azure Blob Storage	JSON handoff and insights archive

Tool	Role in this pipeline
Azure Key Vault	SQL connection string and API keys — never in prompts
BeechTree_LRN (existing SQL)	Source data for Three-Pillar procedures
Entra ID / Managed Identity	Function to SQL/Blob; Logic App to Foundry
Application Insights	Run logs, failures, duration

5.2 Foundry (Agent 2)

Tool	Role in this pipeline
Deployed chat model	Write insights from the JSON
Structured outputs	Force a fixed insights JSON schema
Instructions + diagnostic playbook	How to read LIS / PMS / Cash (funnel, gap, collection rate, write-off ratio)
OpenAPI tool (optional)	Save insights via a small HTTP API

5.3 Already in the LRN repository

Piece	Role in this pipeline
Three BeechTree Three-Pillar stored procedures	Authoritative numbers for Agent 1
ClaimLineCSVDataCapture STEP 16 / STEP 14	“New Executive Summary generated” event
usp_NotesInsight_Insert	Store insights on Executive Summary Notes
ExecSummaryThreePillarViewModel	JSON field map matching the dashboard

5.4 Optional later

Tool	Role
Azure Service Bus	Reliable event instead of a raw HTTP call
Teams or Outlook connector	Notify when insights are ready
API Management	HTTPS + API key in front of the Function

You do not need Playwright, browser login, or Foundry portal workflows for this design. Agent 1 talks to SQL; Agent 2 talks to JSON; Logic Apps sequences them.

6. Build plan — step by step

Work in this order. Each phase has a testable exit. Do not enable the weekly schedule until a manual Logic App run has written Notes rows.

Phase 0 — Azure foundation

1. Create a resource group, for example rg-lrn-threepillar-agents.
2. Create a Foundry resource and project; deploy one mid-size GPT-class model.
3. Create a Storage account and container named three-pillar.
4. Create Key Vault; store BeechTreeConnStr there. Do not put connection strings in agent prompts or git.
5. Create a Function App with Managed Identity; grant Key Vault read, Blob write, and SQL access.

Phase 1 — Agent 1 (JSON exporter)

1. Add a Function, for example POST /api/beech-tree/three-pillar-json.
2. Inputs: lab=Beech_Tree, months=12 (allowed: 3 / 6 / 9 / 12 / 19).
3. Call the three existing stored procedures with the same asOf date and dayWindow the dashboard uses (WeekRange end).
4. Write Blob: beech-tree/three-pillar/{weekFolder}/three-pillar.json.
5. Return { blobUri, generatedAt, lisRows, pmsRows, cashRows } so Logic Apps can validate without opening the file twice.
6. Test once from an HTTP client and confirm JSON counts match the Three-Pillar page for the same months window.

Phase 2 — Fire when Executive Summary is new

1. In ClaimLineCSVDataCapture, after BeechTree Executive Summary refresh succeeds (STEP 16 and STEP 14), POST a small event payload: lab, event=ExecutiveSummaryRefreshed, weekFolder, refreshedAt.
2. Target the Logic App HTTP trigger (swap to Service Bus later if volume or reliability requires it).
3. Send the event only when the ES stored procedures succeeded. Do not start Agent 1 on a failed refresh.

Phase 3 — Agent 2 (Foundry insights)

1. In Foundry, create agent BeechTree-ThreePillar-Insights.
2. Instructions: read LIS / PMS / Cash JSON; do not invent metrics; emit findings with risk (Red / Yellow / Green), evidence (cite JSON fields), and recommended action.
3. Enable structured output using the insights schema in section 4.
4. Ground the agent with Three-Pillar diagnostic rules: sample-to-claim funnel drop, LIS vs PMS reconciliation gap, collection rate, insurance balance composition, write-off ratio, patient collections reality.

Phase 4 — Logic App (schedule and chain)

1. Create Logic App Standard workflow BeechTree-ThreePillar-Pipeline.
2. Attach both triggers to the same workflow: HTTP or Service Bus (primary), and Recurrence (weekly safety net).
3. Action 1: call Agent 1 Function; wait until the Blob exists.
4. Action 2: get Blob content; validate lis, pms, and cash.
5. Action 3: if invalid, fail, alert, and stop.
6. Action 4: Agent action — run Foundry Agent 2; pass the JSON as the user message (do not ask the model to fetch the Blob itself).
7. Action 5: save insights JSON beside the source file in Blob.
8. Action 6: call usp_NotesInsight_Insert (or a small save-insights Function) so notes appear on Executive Summary.
9. Action 7 (optional): email or Teams “insights ready”.
10. Retry Agent 1 and Agent 2 up to three times. Alert if still failing.

Phase 5 — Prove it, then turn on the schedule

1. Manual test: run Agent 1 Function; confirm the Blob.
2. Manual test: run Agent 2 in Foundry with that JSON; confirm the insight schema.
3. Manual test: run the Logic App once; confirm Notes rows on the BeechTree Executive Summary page.
4. Enable the ClaimLineCSVDataCapture webhook.
5. Enable the weekly recurrence last.

7. Trigger versus schedule

Use both. The event is the real weekly clock (data landed). The schedule is only insurance. Agent 2 is never independently scheduled.

Mechanism	Owner	Starts Agent 2?
ES refresh event	ClaimLineCSVDataCapture after successful BeechTree ES SPs	Only after Agent 1 JSON validates
Weekly Logic App recurrence	Logic Apps	Only after Agent 1 JSON validates
Foundry agent timer	Do not use	No — would run without fresh ES data

8. Cost and design notes

- Agent 1 as an Azure Function is cheap and exact. Making Agent 1 a Foundry chat agent adds token cost and can hallucinate numbers.
- Agent 2 is where model tokens are spent. Keep the prompt tight; pass JSON, not screenshots or PDFs of the dashboard.

- Logic Apps Standard plus Foundry is the Microsoft-supported production pattern for event → agent → next agent.
- Foundry portal workflows (visual designer) are not a new-solution option; they retire 1 December 2026. Do not start this project there.
- Pass JSON into Agent 2 as the user message from Logic Apps. That is more reliable than giving Agent 2 a Blob tool and hoping it fetches the right file.
- Reuse usp_NotesInsight_Insert so insights show on the existing Executive Summary Notes & Insights UI instead of a side-channel spreadsheet.

9. First build slice

When implementation starts, deliver these four items before polish (Service Bus, Teams, API Management):

1. Azure Function that writes the Three-Pillar JSON to Blob.
2. Logic App: HTTP trigger → Function → Blob.
3. Foundry Agent 2 that consumes that JSON with structured output.
4. Hook ClaimLineCSVDataCapture STEP 16 / STEP 14 to the Logic App after a successful BeechTree ES refresh.

Decision to confirm at kickoff

Keep Agent 1 as a Function only (recommended), or also register it as a Foundry agent that wraps the same Function via OpenAPI. The Function remains mandatory either way.

10. Appendix — existing refresh points

These are the code locations that already mean “a new Executive Summary was generated” for BeechTree. The webhook in Phase 2 should fire only after these paths succeed.

Path	When	What runs
STEP 16	New RunId or ClaimLineRefresh is true	usp_RefreshBT_ExecutiveSummary and usp_RefreshBT_ExecutiveSummary_LIS_Alt via RefreshExecutiveSummaryByPrefix
STEP 14	No RunId change, but a new TransactionDetail file loaded	usp_RefreshBT_ExecutiveSummary_OnNewFile (BTWOSummary + Executive Summary)

LabMetricsDashboard already maps Three-Pillar result sets in SqlPhiExecutiveSummaryRepository (GetBeechTreeThreePillarLisAsync, GetBeechTreeThreePillarPmsAsync, GetBeechTreeThreePillarCashAsync). Agent 1 should reuse that mapping, either by extracting a shared library or by duplicating the same column-to-JSON contract.

Diagnostic rules Agent 2 should apply

- LIS: sample-to-claim funnel (collected → resulted → billed to insurance); backlog age; % billed of resulted; self-pay / client-bill mix; not resulted.

- PMS: reconciliation gap (PMS billed vs LIS billed to insurance); fully adjusted % and reason Pareto; fully paid %; insurance balance composition; panel avg allowed vs paid; denial rate by carrier.
- Cash: collection rate (fully paid insurance $ ÷ total billed $); partially paid share; insurance balance $ mix (fully denied / no response / partially denied); patient write-off vs balance; patient collection rate; fully adjusted $.

11. Index of terms and tools

Alphabetical index of names used in this plan. Use this with the Contents at the front when handing the PDF to another agent or engineer.

Term	Context
A2A (agent-to-agent)	Lightweight Foundry agent calling; not required for this sequential pipeline.
Agent 1 — JSON Generator	Deterministic Azure Function that calls Three-Pillar SPs and writes Blob JSON.
Agent 2 — Insights	Foundry agent that reads Agent 1 JSON and emits structured findings.
Application Insights	Logging and failure tracking for Function and Logic App runs.
asOf / dayWindow	Comparable-month cutoff from WeekRange end; must match the dashboard.
Azure Blob Storage	Handoff store for three-pillar.json and insights JSON.
Azure Functions	Runtime for Agent 1 exporter API.
Azure Key Vault	Secret store for BeechTree SQL connection string.
Azure Logic Apps Standard	Orchestrator: event, schedule, validate, call Agent 2.
BeechTree_ LRN	Lab SQL database hosting Three-Pillar stored procedures.
BeechTree-ThreePillar-Insights	Suggested Foundry agent name for Agent 2.
BeechTree-ThreePillar-Pipeline	Suggested Logic App workflow name.
ClaimLineCSVDataCapture	Ingestion worker; STEP 14 / STEP 16 are the ES-ready events.
Collection rate	Cash formula: fully paid insurance $ ÷ total billed $.
Entra ID / Managed Identity	Auth from Function and Logic App to SQL, Blob, and Foundry.
ExecSummaryThreePillarViewModel	C# model that defines the JSON field map.
Executive Summary Notes	UI/table destination for Agent 2 via usp_NotesInsight_Insert.
Foundry structured outputs	Forces Agent 2 to return a fixed insights JSON schema.
Foundry visual workflows	Do not use; retiring 1 December 2026. Use Logic Apps instead.
LIS pillar	Sample-to-claim funnel, backlog, billed-of-resulted, self-pay / client bill.
Microsoft Foundry	Host for Agent 2 model, instructions, and optional OpenAPI tools.
OpenAPI tool	Optional Foundry wrapper so an agent can call the Function or save-insights API.
PMS pillar	Reconciliation gap, fully adjusted/paid, denials, maturity, panel averages.
Cash pillar	Dollar leakage: collection, IB mix, patient WO, fully adjusted $.
Service Bus	Optional reliable replacement for the HTTP webhook.
STEP 14	ES refresh when only a new TransactionDetail file arrived.
STEP 16	ES aggregate refresh after a new RunId / claim-line change.
Three-Pillar Diagnostic page	Dashboard URL used to visually verify Agent 1 JSON.

Term	Context
usp_ GetBeechTree_ ThreePillarCashDiagnostic	Cash stored procedure.
usp_ GetBeechTree_ ThreePillarLisDiagnostic	LIS stored procedure.
usp_ GetBeechTree_ ThreePillarPmsDiagnostic	PMS stored procedure.
usp_ NotesInsight_ Insert	Writes an insight row onto Executive Summary Notes.
usp_ RefreshBT_ ExecutiveSummary	BeechTree Executive Summary aggregate refresh.
WeekRange	Billed week folder; end date anchors asOf and dayWindow.

End of document. Next action: confirm Agent 1 as Function-only versus Foundry-wrapped Function, then begin Phase 0.